

# Pseudo-code of the neighborhood generation in the Simulated Annealing algorithm

Supplementary material to the paper  
*Parallel machine scheduling with simultaneous interruptions:  
a case study in shoe sole production*

## Solution representation

A feasible solution is a  $p$ -tuple  $S = (S_1, \dots, S_p)$ , one entry per shelf, where  $S_\ell$  is the ordered sequence of the *sub-jobs* processed on shelf  $\ell$ , in the order in which they are processed. A sub-job is a triple  $(j, i, q)$  formed by a job  $j \in J$  (a shoe size), one of its mold copies  $i \in \{1, \dots, c_j\}$ , and the quantity  $q \in \mathbb{Z}_{\geq 0}$  of soles produced with that copy. We write  $S_\ell[r]$  for the sub-job in position  $r$  of shelf  $\ell$ , and call *position* of  $S$  any pair  $(\ell, r)$  with  $1 \leq \ell \leq p$  and  $1 \leq r \leq |S_\ell|$ .

Every solution handled by the algorithm contains each pair  $(j, i)$  exactly once and satisfies  $\sum_{i=1}^{c_j} q_{j,i} = n_j$  for every job  $j$ , where  $q_{j,i}$  is the quantity of the sub-job  $(j, i)$  and  $n_j$  is the number of soles of size  $j$  required in the order. The initial solution is obtained by drawing, for each job  $j$ , a vector  $(q_1, \dots, q_{c_j})$  uniformly at random among the vectors of  $c_j$  non-negative integers summing to  $n_j$ , and by appending each resulting sub-job to a shelf drawn uniformly at random. Both operators below preserve these two properties, so every candidate solution is feasible and no repair step is needed.

## Neighborhood

A neighbor of the current solution is generated by **TwoStepNeighbor()** (Algorithm 1), which combines two elementary moves: **Relocate()**, which moves a single sub-job to a randomly chosen shelf and position, leaving all quantities unchanged, and **Redistribute()**, which resamples how the quantity of a multi-mold job is divided among its mold copies, leaving all positions unchanged. Three points deserve to be made explicit.

- The sub-job drawn in line 2 is used *only* to decide which of the two moves is applied. Neither branch necessarily acts on it: **Relocate()** and **Redistribute()** draw their own sub-job, respectively their own job, independently of that first draw.
- The name “two-step” refers to the single-mold branch, in which two independent relocations are applied in sequence, the second one acting on the outcome of the first. The multi-mold branch applies a single redistribution and no relocation.
- The quantity  $n_j$  of the selected job is partitioned uniformly at random over all the vectors of  $c_j$  non-negative integers summing to  $n_j$  (line 12), which the implementation samples by the classical stars-and-bars scheme. Zero quantities are admissible, which allows the search to concentrate a job on a subset of its mold copies.

---

**Algorithm 1:** Neighbor generation in the SA algorithm

---

**Input:** Solution  $S$ ; number of shelves  $p$ ; mold availabilities  $\{c_j\}_{j \in J}$

**Output:** A neighbor of  $S$

```
1 Function TwoStepNeighbor( $S$ ):
2   Draw a sub-job of  $S$  uniformly at random and let  $j^*$  be its job ;    // switch only
3   if  $c_{j^*} = 1$  then return Relocate(Relocate( $S$ ));
4   else return Redistribute( $S$ );

5 Function Relocate( $S$ ):
6   Draw a position  $(\ell_s, r_s)$  of  $S$  uniformly at random;  $\sigma \leftarrow S_{\ell_s}[r_s]$ ;
7   Draw  $\ell_t$  uniformly from  $\{1, \dots, p\}$  and  $r_t$  uniformly from  $\{1, \dots, |S_{\ell_t}| + 1\}$  ;
   //  $\ell_t = \ell_s$  allowed
8   Remove  $\sigma$  from  $S_{\ell_s}$ ; if  $\ell_t = \ell_s$  and  $r_t > r_s$  then  $r_t \leftarrow r_t - 1$ ;
9   return the solution obtained by inserting  $\sigma$  in shelf  $\ell_t$  at position  $r_t$  ;
   // quantities unchanged

10 Function Redistribute( $S$ ):
11   Draw a job  $j$  uniformly at random from  $\{j \in J : c_j > 1\}$ ;
12   Draw  $(q_1, \dots, q_{c_j})$  uniformly at random from  $\{q \in \mathbb{Z}_{\geq 0}^{c_j} : \sum_i q_i = n_j\}$ ;
13   return the solution obtained by setting to  $q_i$  the quantity of the sub-job  $(j, i)$ , for
    $i = 1, \dots, c_j$  ;    // positions unchanged
```

---

## Correspondence with the implementation

Algorithm 1 transcribes three functions of the file `simulated_annealing.jl`, in the directory `scripts` of the project repository

<https://github.com/ndlopes-github/ShoeOptSetupTime>

namely:

|                                |                                               |
|--------------------------------|-----------------------------------------------|
| <code>TwoStepNeighbor()</code> | <code>two_step_neighbor</code>                |
| <code>Relocate()</code>        | <code>neighbor_ppartition</code>              |
| <code>Redistribute()</code>    | <code>neighbor_quantity_redistribution</code> |

The uniform draw of line 12, which is also used for the initial solution, is performed by `split_quantity_randomly`, and the initial solution is built by `create_random_partition`, in the same file.